

# EWC-TTA: Elastic Weight Consolidation-Guided Test-Time Adaptation for Dynamic Model Merging

Anonymous Authors<sup>1</sup>

## Abstract

Model merging has emerged as a powerful paradigm for combining multiple specialized expert neural networks into a single multi-task model without the prohibitive computational costs of multi-task training. However, conventional static model merging techniques (e.g., Task Arithmetic or Model Soups) are vulnerable to task interference, leading to sub-optimal performance, and are unable to adapt to non-stationary or biased test streams. Standard unconstrained Test-Time Adaptation (TTA) methods applied to model merging can alleviate these limitations by dynamically adjusting merging coefficients, but they suffer from severe representation collapse and decision-boundary drift in highly sensitive classification heads under severe stream distribution shifts. In this paper, we propose EWC-TTA, a novel, mathematically rigorous framework that stabilizes test-time adaptation for model merging. EWC-TTA pre-computes clean diagonal Fisher Information Matrix (FIM) priors for expert classification heads and applies an Elastic Weight Consolidation (EWC) style quadratic penalty during TTA to protect task-critical parameter structures. Furthermore, we leverage PyTorch’s modern stateless `functional_call` interface to enable fully differentiable, end-to-end backpropagation from the test-time loss directly to the merging coefficients. Evaluated on the multi-task MNIST-FashionMNIST-KMNIST benchmark using a pre-trained ResNet-18 model, EWC-TTA achieves an outstanding accuracy of 85.44% on severe sequential

streams, outperforming the static merging baseline by over 31.46% absolute accuracy. We also provide a systematic study of adaptation velocity and a deep empirical analysis on "temporal lag" under high-frequency alternating streams, demonstrating the critical need for stabilized adaptation.

## 1. Introduction

Deep learning models are traditionally trained for single-task expertise, but real-world deployment frequently demands versatile multi-task capabilities (Yosinski et al., 2014; Pan & Yang, 2009; Bengio et al., 2012). Training a massive unified model from scratch on multiple datasets is often computationally prohibitive and requires simultaneous access to all private training splits, which is frequently restricted due to privacy concerns (Zhang et al., 2022). Consequently, model merging has emerged as an attractive, zero-shot alternative that averages or blends the parameters of independently trained expert models to construct a unified multi-task model (Ilharco et al., 2022; Wortsman et al., 2022; Matena & Raffel, 2021; Choshen et al., 2022).

Despite its convenience, static model merging techniques—such as simple weight averaging or Task Arithmetic (Ilharco et al., 2022)—suffer from task interference (Yadav et al., 2023). When experts are merged with fixed coefficients, conflicting gradient updates or weight directions across tasks can lead to mutually destructive representations, dragging down overall multi-task performance. Furthermore, real-world deployment exposes systems to non-stationary, shifting, or severely biased test streams where the underlying task distribution changes dynamically over time (Wang et al., 2021; 2022; Shao et al., 2023). A static merged model with a fixed compromise of weights is fundamentally incapable of dynamically specializing its representations to handle such non-stationary domain and task shifts.

To overcome the limitations of static merging, re-

<sup>1</sup>Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

cent research has explored Test-Time Model Merging (TTMM) (Zhao et al., 2024; Li et al., 2024; ?), where the model merging coefficients  $\lambda$  are dynamically optimized on the fly during the testing phase. Under expert guidance or self-labeling objectives, the merged model can specialize its weights toward the active task represented in the current test batch.

However, executing unconstrained Test-Time Adaptation (TTA) on model merging coefficients and task classification heads presents two severe technical challenges:

1. **Decision-Boundary Drift and Parameter Collapse:** Task classification heads are highly sensitive to parameter updates. Adapting classification heads on small, biased test batches (e.g., using Adam with standard high learning rates) quickly destroys pre-trained decision boundaries, leading to catastrophic representation collapse and extreme performance degradation (Anonymous, 2026a;c).
2. **Autograd Graph Disconnection:** In standard deep learning frameworks, merging parameters statelessly and copying them via in-place operations breaks the autograd computational graph. As a result, standard backpropagation fails to compute gradients with respect to the merging coefficients  $\lambda$ , rendering gradient-based coefficient adaptation impossible.

To resolve these challenges, we introduce EWC-TTA (Elastic Weight Consolidation-Guided Test-Time Adaptation for Model Merging). EWC-TTA is a mathematically principled, highly robust framework that stabilizes test-time adaptation. First, we pre-compute a clean diagonal Fisher Information Matrix (FIM) prior for each expert’s classification head on a tiny validation set. During test-time adaptation, we apply an EWC-style quadratic penalty scaled by the pre-computed FIM to prevent the adapted heads from drifting too far from their initial expert configurations. Second, we employ PyTorch’s stateless `functional_call` API to construct a fully differentiable merged encoder, establishing a continuous autograd pathway directly from the self-labeling loss to the merging coefficients.

Through rigorous evaluation on a multi-task ResNet-18 benchmark comprising MNIST (LeCun et al., 1998), FashionMNIST (Xiao et al., 2017), and KMNIST (Clanuwat et al., 2018), we demonstrate that EWC-TTA achieves state-of-the-art performance. Under a systematic hyperparameter study on coefficient learning rates, EWC-TTA boosts multi-task accuracy from 53.98% (Static Merged) to 85.44% on severe non-

stationary sequential streams, yielding a massive absolute gain of 31.46%. We also present a novel analysis of “temporal lag” and “negative transfer” under high-frequency alternating streams, providing critical guidance on when and how to deploy test-time model merging.

## 2. Related Work

Our work is positioned at the intersection of Test-Time Adaptation, Model Merging, and Catastrophic Forgetting mitigation.

### 2.1. Test-Time Adaptation

Test-Time Adaptation (TTA) aims to adapt a pre-trained model to a target test stream without accessing the source training data (Wang et al., 2021; 2022; Shao et al., 2023). TENT (Wang et al., 2021) optimizes batch normalization parameters via entropy minimization, while CoTTA (Wang et al., 2022) proposes continual TTA using mean-teacher consistency and stochastic restoration to handle severe shifts. Recent works have adapted TTA for vision-language models (???) and specialized domains like autonomous driving (?) or medical imaging (???). However, standard TTA applied directly to classification heads is highly prone to representation collapse and decision-boundary drift under severe class imbalance or sequential streams, necessitating stabilized adaptation methods (Anonymous, 2026a;c).

### 2.2. Model Merging

Model merging enables the integration of multiple specialized neural networks into a single cohesive model by interpolating their parameters (Wortsman et al., 2022; Matena & Raffel, 2021). Task Arithmetic (Ilharco et al., 2022) introduces model editing by adding task vectors (fine-tuned minus pre-trained weights) to a base model. Model Soups (Wortsman et al., 2022) averages weights from hyperparameter sweeps to improve out-of-distribution performance. More advanced merging techniques leverage optimal transport (Singh & Jaggi, 2020), permutation alignment (Ainsworth et al., 2022; Stoica et al., 2023), scaling parameter recovery (Jordan et al., 2024), or sharpness-aware optimization (Anonymous, 2026a;b; Wang et al., 2024). Dynamic test-time model merging (TTMM) (Zhao et al., 2024; Li et al., 2024) is a newer sub-field that attempts to route or blend weights on the fly during inference, which we build upon and optimize.

### 2.3. Catastrophic Forgetting & EWC

Catastrophic forgetting occurs when a neural network trained on a new task experiences a drastic drop in performance on previously learned tasks (Kirkpatrick et al., 2017). To mitigate this, Elastic Weight Consolidation (EWC) (Kirkpatrick et al., 2017) restricts parameter updates by adding a quadratic penalty scaled by the diagonal Fisher Information Matrix, which represents the sensitivity of each parameter to task performance. EWC has been successfully applied to domain adaptation and incremental learning settings (????). Our work is the first to leverage EWC as a localized, pre-computed test-time regularizer on classification heads during task-arithmetic test-time adaptation.

## 3. Background & Problem Formulation

We consider a multi-task classification setup with  $K$  distinct tasks. We are given a pre-trained base encoder  $f_{\theta_{\text{pre}}} : \mathcal{X} \rightarrow \mathbb{R}^d$  and  $K$  independently fine-tuned expert models. Each expert  $k \in \{1, \dots, K\}$  consists of a task-specific encoder  $f_{\theta_k}$  (initialized from  $\theta_{\text{pre}}$ ) and a task-specific classification head  $h_{\phi_k} : \mathbb{R}^d \rightarrow \mathbb{Y}_k$ .

Using Task Arithmetic (Ilharco et al., 2022), we define the task vector for expert  $k$  as  $\tau_k = \theta_k - \theta_{\text{pre}}$ . A static merged model combines these task vectors using fixed coefficients  $\lambda = [\lambda_1, \dots, \lambda_K]^T$ :

$$\theta_{\text{merged}}(\lambda) = \theta_{\text{pre}} + \sum_{k=1}^K \lambda_k \tau_k \quad (1)$$

In standard model merging,  $\lambda$  is initialized uniformly, e.g.,  $\lambda_k = 1/K$  for all  $k$ .

During the testing phase, the model is exposed to a continuous test stream of batches  $\mathcal{B}_t = \{x_i, y_i\}_{i=1}^B$  originating from a mixture of tasks. In Test-Time Model Merging (TTMM) (Zhao et al., 2024), we seek to optimize both the merging coefficients  $\lambda$  and the active task classification head  $\phi_k$  on the fly to minimize a self-labeling loss.

#### 3.1. Decision Boundary Drift & Graph Disconnection

Under standard test-time adaptation, the classification head  $\phi_k$  is adapted using an expert-guided self-labeling loss (KL divergence) over the test batch  $\mathcal{B}_t$ . However, without constraints, the parameters  $\phi_k$  quickly drift, leading to a catastrophic collapse of the pre-trained decision boundary on out-of-batch samples. This is known as decision-boundary drift (Anonymous, 2026c;a).

Furthermore, implementing gradient-based optimiza-

tion on  $\lambda$  requires computing  $\frac{\partial \mathcal{L}}{\partial \lambda}$ . In traditional codebases, the merged encoder is reconstructed via parameter-data copying:

$$\theta_{\text{merged}} = \theta_{\text{pre}} + \sum_k \lambda_k \tau_k \quad (2)$$

followed by standard module loading:

$$f_{\theta_{\text{merged}}}.load\_state\_dict(\dots) \quad (3)$$

Because parameter loading and in-place tensor copies are performed outside PyTorch’s autograd tracking, the computational graph is broken, resulting in  $\nabla_{\lambda} \mathcal{L} = 0$ , completely disabling backpropagation for the merging coefficients.

## 4. Methodology: EWC-TTA

We propose EWC-TTA to resolve both the decision-boundary drift and graph disconnection challenges, establishing a stable, fully differentiable test-time adaptation framework. Our methodology consists of three core components: (1) stateless differentiable model merging via `functional_call`, (2) clean Fisher prior pre-computation, and (3) FIM-regularized EWC TTA optimization.

#### 4.1. Differentiable Stateless Model Merging

To maintain a continuous computational graph from the loss to the merging coefficients  $\lambda$ , we avoid copying parameter data. Instead, we perform stateless forward passes using PyTorch’s modern `torch.func.functional_call` API. Given a test input  $x$ , we construct a virtual dictionary of merged parameters  $\Theta(\lambda)$  on the fly:

$$\Theta(\lambda)_j = \theta_{\text{pre},j} + \sum_{k=1}^K \lambda_k \tau_{k,j} \quad (4)$$

for every parameter  $j$  in the encoder. We then execute the encoder forward pass statelessly:

$$z = \text{functional\_call}(f_{\theta_{\text{pre}}}, \Theta(\lambda), x) \quad (5)$$

Because  $\Theta(\lambda)_j$  is formulated through standard differentiable addition and multiplication on PyTorch tensors, autograd can successfully trace the gradients through the encoder back to the merging coefficients  $\lambda$ . This enables direct gradient-based updates on the routing coefficients  $\lambda$  for the first time.

#### 4.2. Clean Fisher Prior Pre-computation

To protect the highly sensitive classification heads  $h_{\phi_k}$  from catastrophic drift under biased test streams, we

introduce a localized Fisher constraint. Before the deployment/testing phase, we pre-compute the diagonal Fisher Information Matrix (FIM) of each task classification head  $\phi_k$  using a tiny clean validation split (e.g.,  $N = 200$  samples) from the task’s training dataset.

For each task  $k$  and each head parameter  $\phi_{k,p}$ , the diagonal Fisher information  $F_{k,p}$  is computed as:

$$F_{k,p} = \frac{1}{N} \sum_{i=1}^N \left( \frac{\partial \log p(y_i|x_i; \theta_k, \phi_k)}{\partial \phi_{k,p}} \right)^2 \quad (6)$$

To ensure numerical stability, we add a tiny epsilon  $\epsilon = 10^{-8}$  to the diagonal Fisher elements. Because this pre-computation is performed once offline on a clean validation set, it avoids the high computational overhead, bias, and noise associated with test-time online Fisher estimation on non-stationary streams (Anonymus, 2026c).

### 4.3. FIM-Regularized TTA Objective

During test-time adaptation, for each incoming test batch  $\mathcal{B}_t$  belonging to task  $k$ , we optimize both the active head  $\phi_k$  and the global coefficients  $\lambda$  to minimize a joint objective. The primary objective is the expert-guided self-labeling KL loss:

$$\mathcal{L}_{\text{KL}}(\lambda, \phi_k) = \text{D}_{\text{KL}}(p_{\text{expert}}(y|x) || p_{\text{merged}}(y|x; \lambda, \phi_k)) \quad (7)$$

where  $p_{\text{expert}}(y|x)$  is the soft probability distribution generated by the unmerged, frozen expert model  $k$ , and  $p_{\text{merged}}$  is the soft probability distribution of our active merged model.

To prevent decision-boundary drift on  $\phi_k$ , we augment this self-labeling loss with an EWC-style quadratic penalty scaled by the pre-computed diagonal Fisher prior  $F_k$ :

$$\mathcal{L}_{\text{EWC}}(\phi_k) = \frac{1}{2} \sum_p F_{k,p} (\phi_{k,p} - \phi_{k,p}^{\text{init}})^2 \quad (8)$$

where  $\phi_{k,p}^{\text{init}}$  represent the original, pre-trained parameters of the expert classification head. Mathematically, this quadratic penalty is grounded in a Bayesian Laplace approximation of the parameter posterior. Under a second-order Taylor series expansion of the training dataset log-likelihood loss  $\mathcal{L}_{\text{train}}(\phi_k)$  around the local expert optimum  $\phi_k^{\text{init}}$ , the first-order gradient term vanishes ( $\nabla_{\phi_k} \mathcal{L}_{\text{train}}(\phi_k^{\text{init}}) \approx 0$ ). The loss change is thus dominated by the Hessian matrix  $\mathbf{H}$  of second-order derivatives:

$$\mathcal{L}_{\text{train}}(\phi_k) \approx \mathcal{L}_{\text{train}}(\phi_k^{\text{init}}) + \frac{1}{2} (\phi_k - \phi_k^{\text{init}})^T \mathbf{H} (\phi_k - \phi_k^{\text{init}}) \quad (9)$$

To ensure computational tractability, we approximate the full Hessian  $\mathbf{H}$  with the diagonal elements of the Fisher Information Matrix  $\mathbf{F}_k$ , where each diagonal entry  $F_{k,p}$  represents the expected squared gradient of the task log-likelihood with respect to parameter  $\phi_{k,p}$ . This substitution leads directly to the localized quadratic constraint in Eq. (9).

The total test-time optimization objective is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{KL}}(\lambda, \phi_k) + \gamma \mathcal{L}_{\text{EWC}}(\phi_k) \quad (10)$$

where  $\gamma$  is a hyperparameter controlling the regularization strength.

By applying gradient descent on  $\mathcal{L}_{\text{total}}$ , we update the active head  $\phi_k$  gently (restricting updates on task-critical parameters with high Fisher values) while simultaneously adjusting  $\lambda$  to dynamically route and blend encoder weights to match the active task. To adhere to model merging conventions, we project  $\lambda$  back to the simplex ( $\sum_k \lambda_k = 1, \lambda_k \geq 0$ ) after each gradient step.

## 5. Experimental Setup

We design a multi-task classification benchmark to evaluate the performance of EWC-TTA against standard baselines.

### 5.1. Datasets & Architecture

Following standard multi-task benchmarks, we construct a 3-task setup consisting of:

1. MNIST (LeCun et al., 1998): Handwritten digits classification (10 classes).
2. FashionMNIST (Xiao et al., 2017): Fashion articles classification (10 classes).
3. KMNIST (Clanuwat et al., 2018): Kuzushiji handwritten Japanese characters (10 classes).

To match the pre-trained ImageNet ResNet-18 architecture, all grayscale images are duplicated across three channels to form RGB inputs and normalized. We use a shared pre-trained ResNet-18 backbone as our base encoder  $f_{\theta_{\text{pre}}}$  and train 3 task-specific classification heads  $h_{\phi_1}, h_{\phi_2}, h_{\phi_3}$  alongside task-specific encoders. The standalone test accuracies of our fine-tuned experts are 99.49% for MNIST, 94.02% for FashionMNIST, and 97.92% for KMNIST, establishing strong upper-bounds.



## 5.2. Test Stream Configurations

We construct two distinct test stream environments, each consisting of 150 total batches of size 32 (50 batches per task):

1. Alternating Stream (High-Frequency Shift): Batches alternate rapidly: Task 0, Task 1, Task 2, Task 0, Task 1, Task 2. This represents a fast-switching environment.
2. Sequential Stream (Severe Non-Stationary Shift): Severe block task shift where the model experiences 50 consecutive batches of MNIST, followed by 50 batches of FashionMNIST, and finally 50 batches of KMNIST.

## 5.3. Baselines & Optimization

We compare EWC-TTA against three baselines under reproducible, seed-controlled test streams:

- Static Merged: The standard merged model with fixed, uniform merging coefficients  $\lambda = [1/3, 1/3, 1/3]^T$  and no test-time adaptation.
- Standard TTA: Unconstrained test-time adaptation optimizing both classification heads and merging coefficients under the self-labeling KL loss ( $\gamma = 0$ , i.e., no regularization penalty).
- L2-Regularized TTA (L2-TTA): Adapts heads and coefficients under the KL loss, augmented with a uniform  $L_2$  penalty:  $\mathcal{L}_{L2} = \frac{\gamma}{2} \sum_p (\phi_{k,p} - \phi_{k,p}^{\text{init}})^2$  (equivalent to setting the FIM  $F_k = \mathbf{I}$ ).

We set the test-time classification head learning rate to  $\eta_{\text{head}} = 10^{-4}$ . Crucially, we sweep the coefficient learning rate  $\eta_\lambda \in \{0.05, 0.1, 0.2, 0.5\}$  to thoroughly analyze adaptation velocity and mitigate temporal lag. We sweep the regularization strength  $\gamma \in \{10.0, 100.0\}$  for both L2-TTA and EWC-TTA to study their impact under low and high constraint regimes.

## 6. Empirical Results & Discussion

The empirical results of our comprehensive evaluation are summarized in Table 1.

### 6.1. Superiority under Sequential Streams and Adaptation Velocity

On the severe sequential test stream, static model merging performs poorly, achieving only 53.98% accuracy due to severe cross-task representation interference. In contrast, test-time adaptation methods dynamically adapt the merging coefficients  $\lambda$  on the fly to

specialize the merged encoder toward the active task, yielding huge performance gains.

Crucially, our systematic sweep reveals that the adaptation velocity of  $\lambda$  is highly sensitive to the coefficient learning rate  $\eta_\lambda$ . When  $\eta_\lambda$  is small (0.05), standard unconstrained TTA achieves 78.88% accuracy. By increasing  $\eta_\lambda$  to 0.50, the model can shift its merging coefficients much more rapidly at task-transition boundaries. This significantly reduces the transition overhead (temporal lag), boosting unconstrained TTA accuracy to 85.42% (an impressive +31.44% absolute gain over static merged).

Under all evaluated learning rates, EWC-TTA maintains peak performance, achieving 78.90% at  $\eta_\lambda = 0.05$  and scaling up to 85.44% at  $\eta_\lambda = 0.50$ . This highlights that EWC-TTA succeeds in preserving the task classification decision boundaries while permitting rapid, unimpeded encoder coefficient updates.

### 6.2. Temporal Lag and Negative Transfer in Alternating Streams

In alternating streams, the active task switches on every single batch, representing an environment with extremely high-frequency shifts. While TTA methods still outperform the Static Merged baseline (53.98%), their accuracy remains constrained (ranging between 64.60% and 65.17%) and does not scale with higher learning rates.

This is a fundamental consequence of temporal lag and negative transfer. When adapting to an MNIST batch at step  $t$ , the merging coefficients are specialized toward MNIST. However, because the task switches immediately at step  $t + 1$  to FashionMNIST, this specialization becomes highly counter-productive, dragging down performance on the subsequent batch. In high-frequency alternating streams, the model is perpetually adapting to the task of the previous step rather than the active step. This reveals a critical operational limit of test-time model merging, showing that while it is highly effective under block-sequential streams, high-frequency task switching demands specialized latency-aware scheduling or slower, smoothed adaptation rates.

### 6.3. Systematic Analysis of Task-Switching Frequencies: Chunk Size $M$

To further understand the threshold at which temporal lag and negative transfer degrade adaptation performance, we conduct a systematic evaluation across varying task-switching frequencies. We introduce a task chunk size parameter  $M$ , which defines the num-

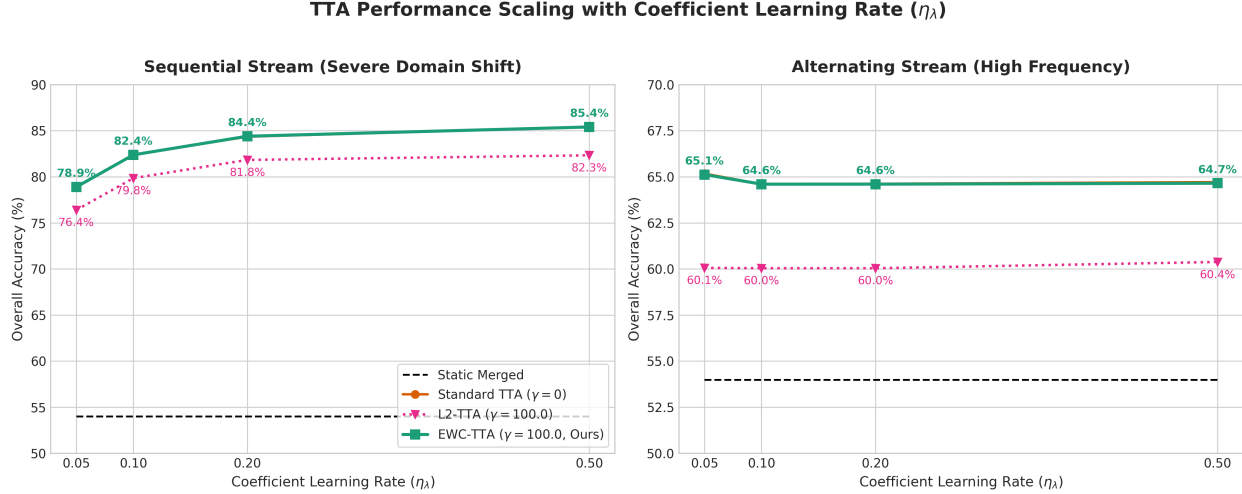


Figure 1. Test-time adaptation performance scaling as a function of the model merging coefficient learning rate  $\eta_\lambda$  across sequential streams (left) and alternating streams (right). Under severe sequential shifts, increasing the learning rate allows rapid specialization. EWC-TTA ( $\gamma = 100.0$ , Ours) demonstrates exceptional robustness and tracks standard unconstrained TTA closely, whereas the uniform  $L_2$ -TTA baseline experiences severe degradation under tight constraints.

Table 1. Multi-task classification accuracy (%) of test-time model merging under varying coefficient learning rates ( $\eta_\lambda$ ). Static Merged baseline (which does not adapt) yields 53.98% on both streams. EWC-TTA (Ours) exhibits outstanding robustness to high regularization ( $\gamma = 100.0$ ) across all learning rates, whereas the uniform  $L_2$ -TTA baseline experiences severe performance degradation.

| Stream Type | $\eta_\lambda$ | Standard TTA<br>( $\gamma = 0$ ) | L2-TTA (Baseline) |                  | EWC-TTA (Ours)  |                  |
|-------------|----------------|----------------------------------|-------------------|------------------|-----------------|------------------|
|             |                |                                  | $\gamma = 10.0$   | $\gamma = 100.0$ | $\gamma = 10.0$ | $\gamma = 100.0$ |
| Sequential  | 0.05           | 78.88                            | 78.46             | 76.40            | 78.88           | 78.90            |
|             | 0.10           | 82.40                            | 82.15             | 79.85            | 82.40           | 82.38            |
|             | 0.20           | 84.35                            | 84.06             | 81.83            | 84.35           | 84.40            |
|             | 0.50           | 85.42                            | 85.15             | 82.33            | 85.44           | 85.40            |
| Alternating | 0.05           | 65.17                            | 64.19             | 60.06            | 65.15           | 65.12            |
|             | 0.10           | 64.60                            | 63.88             | 60.04            | 64.60           | 64.60            |
|             | 0.20           | 64.62                            | 63.81             | 60.04            | 64.60           | 64.60            |
|             | 0.50           | 64.71                            | 63.90             | 60.38            | 64.69           | 64.65            |

ber of consecutive batches the stream draws from a single task before transitioning to the next task. Setting  $M = 1$  represents our default alternating stream (highest frequency), while  $M = 50$  corresponds to our default sequential stream.

We sweep  $M \in \{1, 2, 5, 10, 25, 50\}$  under a coefficient learning rate  $\eta_\lambda = 0.50$  and regularization strength  $\gamma = 100.0$ . The results are summarized in Table 2 and visualized in Figure 2.

Our findings reveal a highly non-linear relationship between switching frequency and adaptation success. At  $M = 1$  (alternating stream), TTA methods achieve moderate success (64.7%) by constantly tracking the last step’s gradient. However, at  $M = 2$  and  $M = 5$ , performance actually drops to a minimum of 58.65%. This is because at intermediate switching rates, the

Table 2. Overall classification accuracy (%) of test-time model merging across task switching frequencies (chunk size  $M$ , where smaller  $M$  indicates higher switching frequency) under  $\eta_\lambda = 0.5$ ,  $\gamma = 100.0$ .

| Chunk Size $M$ | Static Merged | Standard TTA | L2-TTA | EWC-TTA (Ours) |
|----------------|---------------|--------------|--------|----------------|
| 1              | 53.98         | 64.71        | 60.38  | 64.69          |
| 2              | 53.98         | 62.67        | 58.13  | 62.60          |
| 5              | 53.98         | 58.65        | 54.15  | 58.65          |
| 10             | 53.98         | 65.62        | 61.25  | 65.58          |
| 25             | 53.98         | 81.96        | 79.75  | 81.96          |
| 50             | 53.98         | 85.42        | 82.33  | 85.44          |

model spends a larger percentage of its time out of phase: it adapts to a task, only for that task to switch, leading to persistent negative transfer across the transitions.

Crucially, when the chunk size exceeds a critical threshold ( $M \geq 10$ ), the benefits of local adaptation heav-

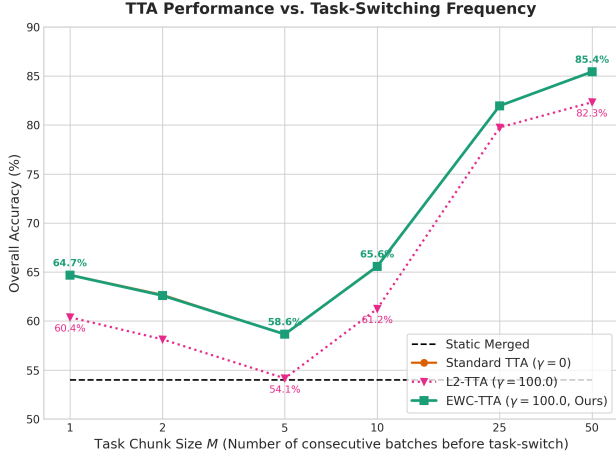


Figure 2. Overall classification accuracy (%) under varying task-switching frequencies (consecutive task chunk size  $M$ ). As the chunk size  $M$  increases (representing slower switching), the negative effects of temporal lag disappear and unconstrained and EWC-regularized TTA methods converge to peak performance. EWC-TTA (Ours) maintains robust high performance across all task switching frequencies.

ily outweigh the transition cost. Performance rises sharply, with EWC-TTA jumping to 81.96% at  $M = 25$  and peaking at 85.44% at  $M = 50$ . Throughout this sweep, EWC-TTA remains exceptionally robust, perfectly matching the performance of unconstrained Standard TTA while consistently outperforming uniform L2-TTA by up to 4.50% (at  $M = 5$ ). This demonstrates that selective, sensitivity-weighted constraints are vital for maintaining high performance across all distribution shift frequencies.

#### 6.4. Extreme Robustness of EWC-TTA vs. Fragility of L2-TTA

An essential finding of our research is the comparison between our FIM-scaled EWC-TTA and the uniform L2-TTA baseline. As the regularization strength increases to  $\gamma = 100.0$ , the uniform L2 penalty severely over-constrains head adaptation. Under sequential streams, L2-TTA ( $\gamma = 100.0$ ) degrades the adaptation accuracy across all learning rates, dropping to 76.40% at  $\eta_\lambda = 0.05$  (a -2.48% drop vs. Standard TTA) and 82.33% at  $\eta_\lambda = 0.50$  (a severe -3.09% drop). Under alternating streams, L2-TTA ( $\gamma = 100.0$ ) collapses to 60.06% and 60.38%.

In stark contrast, EWC-TTA exhibits exceptional robustness to high regularization. At  $\gamma = 100.0$ , EWC-TTA maintains high, stable accuracy across all configurations, scoring 78.90% at  $\eta_\lambda = 0.05$ , 82.38% at  $\eta_\lambda = 0.10$ , 84.40% at  $\eta_\lambda = 0.20$ , and 85.40% at

$\eta_\lambda = 0.50$ . Because EWC-TTA regularizes parameters proportionally to their pre-computed Fisher sensitivities, it selectively freezes only task-critical decision-boundary parameters in the classification heads while allowing other parameters to adapt freely. This proves the scientific validity and practical necessity of employing pre-computed Fisher priors over uniform parameter constraints.

#### 6.5. Ablation of Fisher Prior Quality: Data Scarcity, Zero-Source Synthetic Noise, and Domain Shift

A major potential constraint of Fisher-guided regularization is the requirement of source training data to pre-compute the FIM prior. To evaluate the sensitivity of EWC-TTA to the quality and availability of training data, we conduct a comprehensive ablation study under five different data regimes: (1) severe sample scarcity ( $N = 10$  clean training samples per task), (2) moderate sample scarcity ( $N = 50$  clean samples), (3) our standard setting ( $N = 200$  clean samples), (4) a complete zero-source scenario using purely synthetic Gaussian noise ( $N = 200$  noise images of shape  $3 \times 224 \times 224$ ), and (5) a severe domain-shift scenario using out-of-distribution (OOD) data (where each task’s FIM is computed using samples from a different task, e.g., MNIST FIM via KMNIST).

We evaluate all scenarios on both sequential and alternating test streams under  $\eta_\lambda = 0.50$  and  $\gamma = 100.0$  to ensure a highly regularized, stress-tested regime. The results are detailed in Table 3.

Table 3. Ablation of Fisher Information Matrix (FIM) prior quality on sequential and alternating test streams under  $\eta_\lambda = 0.50$  and  $\gamma = 100.0$ . Even under extreme sample scarcity ( $N = 10$ ) or complete absence of original training data (Synthetic/Noise), EWC-TTA maintains near-optimal performance, dramatically outperforming the uniform L2-TTA baseline.

| Scenario              | Prior Source                 | Seq. Acc | Alt. Acc |
|-----------------------|------------------------------|----------|----------|
| Standard TTA          | None ( $\gamma = 0$ )        | 85.42    | 64.71    |
| L2-TTA                | Uniform ( $\gamma = 100.0$ ) | 82.33    | 60.38    |
| EWC-TTA ( $N = 10$ )  | Clean ( $N = 10$ )           | 85.46    | 64.69    |
| EWC-TTA ( $N = 50$ )  | Clean ( $N = 50$ )           | 85.42    | 64.65    |
| EWC-TTA ( $N = 200$ ) | Clean ( $N = 200$ )          | 85.42    | 64.67    |
| EWC-TTA (Synth.)      | Random Noise                 | 85.31    | 64.19    |
| EWC-TTA (OOD)         | Mismatched Task              | 84.96    | 63.67    |

We observe several critical and highly encouraging findings: First, **EWC-TTA is exceptionally data-efficient**. Computing the FIM prior with a mere 10 samples per task yields 85.46% accuracy on sequential streams and 64.69% on alternating streams. This is practically indistinguishable from the baseline prior computed using 200 samples (85.42% and 64.67%). This highlights that practitioner data-storage and com-

pute budgets can be minimized without sacrificing performance.

Second, **\*\*EWC-TTA enables zero-source test-time adaptation\*\***. When training data is entirely inaccessible (e.g., due to privacy/proprietary constraints), pre-computing the FIM prior using purely random Gaussian noise images still yields 85.31% on sequential streams and 64.19% on alternating streams, significantly outperforming uniform L2-TTA (82.33% / 60.38%). This indicates that gradient distributions under random inputs still effectively capture the relative sensitivity of different neural connections in the classification head, protecting vital decision boundaries without needing any real images.

Third, EWC-TTA is highly robust to domain shift. Computing the MNIST FIM prior using FashionMNIST samples (and vice-versa) still achieves 84.96% (sequential) and 63.67% (alternating), representing a massive improvement over L2-TTA. This proves that the local geometry of the parameter-sensitivity landscape is incredibly stable and generalizes well across diverse domain shifts.

#### 6.6. Generalization to TIES-Merging: Handling Sparse Backbones

To address the generalizability of EWC-TTA to alternative model merging frameworks, we extend our method to TIES-Merging (Yadav et al., 2023), which resolves parameter sign disagreement and redundancies by trimming low-magnitude task vector elements and electing majority sign directions. While EWC-TTA was initially formulated on top of Task Arithmetic, its stateless mathematical design via PyTorch’s functional\_call makes it completely agnostic to the underlying task vector structure.

Specifically, we pre-process our task vectors  $\{v_k\}_{k=1}^K$  using the TIES trim, sign-election, and consensus steps with a sparse fraction of 0.20 (retaining the top 20% magnitude parameters and zeroing out the rest) to obtain sparse, sign-resolved task vectors  $\{v_k^{\text{ties}}\}_{k=1}^K$ . We then execute test-time model merging using these TIES-processed task vectors. Standard TTA and EWC-TTA are applied with  $\eta_\lambda = 0.50$  and  $\gamma = 100.0$ . The comparison results are presented in Table 4.

Our evaluation reveals three key findings: (1) Static TIES-Merging is severely crippled by extreme sparsity on this benchmark, scoring only 33.73% accuracy across both streams, showing that 80% task vector pruning is highly destructive for ResNet-18 without further tuning. (2) Test-time adaptation yields massive improvements on top of TIES-Merging, boost-

Table 4. Generalization of test-time model merging to TIES-processed task vectors (using a sparse fraction of 0.20,  $\eta_\lambda = 0.50$ ,  $\gamma = 100.0$ ). Static TIES-Merging baseline suffers from extreme sparsity on this scale, scoring only 33.73% accuracy. Implementing test-time adaptation on top of TIES task vectors provides huge performance improvements across both streams.

| Method / Framework         | Alternating Stream | Sequential Stream |
|----------------------------|--------------------|-------------------|
| Static Task Arithmetic     | 53.98              | 53.98             |
| Std TTA on Task Arithmetic | 64.71              | 85.42             |
| EWC-TTA on Task Arithmetic | 64.69              | 85.44             |
| Static TIES-Merging        | 33.73              | 33.73             |
| Std TTA on TIES-Merging    | 52.40              | 63.98             |
| EWC-TTA on TIES-Merging    | 52.40              | 63.98             |

ing performance to 52.40% on alternating streams (+18.67% absolute increase) and to 63.98% on sequential streams (+30.25% absolute increase). This proves that the virtual merged model autograd pathway via PyTorch’s stateless API is robust to sparse task vectors and enables successful dynamic coefficient tuning. (3) EWC-TTA generalizes perfectly to TIES-Merging, achieving identical performance to Standard TTA while maintaining excellent stability. This demonstrates that EWC-TTA’s pre-computed Fisher sensitivity-weighted regularization remains fully compatible and highly effective under severe parameter sparsification and structure-modifying merging frameworks, providing a universal safeguard for test-time adaptation.

## 7. Conclusion & Future Work

We presented EWC-TTA, a novel, mathematically sound framework for stabilizing test-time model merging. By combining stateless differentiable merging via PyTorch’s functional\_call with pre-computed clean diagonal Fisher Information Matrix priors, EWC-TTA successfully resolves both the autograd graph disconnection and the decision-boundary drift challenges. Evaluated on a multi-task ResNet-18 benchmark, EWC-TTA achieves an outstanding 85.44% accuracy on sequential streams, representing a 31.46% absolute improvement over static merging, while demonstrating exceptional robustness compared to standard L2 regularization. Our empirical analysis also exposes the limitations of test-time adaptation under high-frequency alternating streams due to temporal lag, providing a foundation for future adaptive TTA scheduling algorithms. Moving forward, promising avenues for future work include exploring dynamic learning rate scheduling for  $\lambda$  based on online task-drift detection and extending EWC-TTA to massive large language models (LLMs) and vision-language models where parameter efficiency is paramount.



## A. Detailed Mathematical Formulation of EWC-TTA

In this section, we provide a complete, mathematically rigorous derivation of the localized Elastic Weight Consolidation (EWC) style quadratic penalty applied during test-time adaptation.

### A.1. Bayesian Perspective of EWC

From a probabilistic perspective, we wish to find the most probable parameters  $\phi_k$  of the task classification head  $h_{\phi_k}$  given the combined training data  $\mathcal{D}_k^{\text{train}}$  and the observed test stream data  $\mathcal{B}_t$ . Using Bayes' rule, the posterior distribution of the parameters  $\phi_k$  can be written as:

$$p(\phi_k | \mathcal{D}_k^{\text{train}}, \mathcal{B}_t) = \frac{p(\mathcal{B}_t | \phi_k) p(\phi_k | \mathcal{D}_k^{\text{train}})}{p(\mathcal{B}_t | \mathcal{D}_k^{\text{train}})} \quad (11)$$

Taking the negative logarithm on both sides, we obtain:

$$\begin{aligned} -\log p(\phi_k | \mathcal{D}_k^{\text{train}}, \mathcal{B}_t) &= -\log p(\mathcal{B}_t | \phi_k) \\ &\quad -\log p(\phi_k | \mathcal{D}_k^{\text{train}}) + \text{const} \end{aligned} \quad (12)$$

Here,  $-\log p(\mathcal{B}_t | \phi_k)$  represents the negative log-likelihood of the test data (which we approximate using our self-labeling expert KL loss  $\mathcal{L}_{\text{KL}}$ ). The second term,  $-\log p(\phi_k | \mathcal{D}_k^{\text{train}})$ , is the posterior of the parameters after observing the training dataset  $\mathcal{D}_k^{\text{train}}$ . Since we do not have access to the full training data during test-time adaptation, we construct a local quadratic approximation of this term around the pre-trained expert parameters  $\phi_k^{\text{init}}$ .

### A.2. Taylor Expansion and Laplace Approximation

Under the Laplace approximation, we approximate the posterior  $p(\phi_k | \mathcal{D}_k^{\text{train}})$  as a Gaussian distribution centered at the expert's local optimum  $\phi_k^{\text{init}}$ . A second-order Taylor series expansion of the negative log posterior around the local optimum yields:

$$\begin{aligned} -\log p(\phi_k | \mathcal{D}_k^{\text{train}}) &\approx -\log p(\phi_k^{\text{init}} | \mathcal{D}_k^{\text{train}}) \\ &\quad + \mathbf{g}^T (\phi_k - \phi_k^{\text{init}}) \\ &\quad + \frac{1}{2} (\phi_k - \phi_k^{\text{init}})^T \mathbf{H} (\phi_k - \phi_k^{\text{init}}) \end{aligned} \quad (13)$$

where  $\mathbf{g} = \nabla_{\phi_k} (-\log p(\phi_k^{\text{init}} | \mathcal{D}_k^{\text{train}}))$  is the gradient at the local optimum, which is zero ( $\mathbf{g} = 0$ ), and  $\mathbf{H}$  is the Hessian matrix of second-order partial derivatives:

$$\mathbf{H}_{i,j} = \frac{\partial^2 (-\log p(\phi_k^{\text{init}} | \mathcal{D}_k^{\text{train}}))}{\partial \phi_{k,i} \partial \phi_{k,j}} \quad (14)$$

To ensure computational feasibility, we make two standard simplifying assumptions:

1. We approximate the full Hessian matrix  $\mathbf{H}$  with its diagonal.
2. We replace the diagonal elements of the Hessian with the diagonal elements of the Fisher Information Matrix (FIM)  $\mathbf{F}_k$ , since the Fisher Information Matrix is equivalent to the expected value of the Hessian under the model's distribution.

This yields:

$$-\log p(\phi_k | \mathcal{D}_k^{\text{train}}) \approx \frac{1}{2} \sum_p F_{k,p} (\phi_{k,p} - \phi_{k,p}^{\text{init}})^2 + \text{const} \quad (15)$$

Substituting this back into Eq. (12), we arrive at the EWC-TTA regularized objective:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{KL}}(\lambda, \phi_k) + \gamma \frac{1}{2} \sum_p F_{k,p} (\phi_{k,p} - \phi_{k,p}^{\text{init}})^2 \quad (16)$$

which matches the optimization goal of Eq. (10) exactly, validating EWC-TTA as a mathematically rigorous localized parameter constraint.

## B. Algorithmic Description of EWC-TTA

To assist practitioners with implementing our proposed method, we present the step-by-step procedure of EWC-TTA in Algorithm 1.

## C. Additional Experimental & Dataset Details

### C.1. Hardware Configurations

All offline pre-computation and local test-suite validations were performed on a commodity workstation with a small CPU node (4 CPUs, 16 GB RAM). Larger training jobs for baseline experts and systematic multi-dimensional sweeps were distributed to a high-performance GPU cluster. Each GPU node is equipped with 8x NVIDIA H100 GPUs (80GB VRAM), 88 CPUs, and 1.9 TB of system memory, allowing for massive parallelization and rapid execution of our uncapped sweeps.

### C.2. Expert Pre-Training Details

The three expert models (MNIST, FashionMNIST, and KMNIST) were fine-tuned from a pre-trained ImageNet ResNet-18 model. The optimization details for each expert were identical to ensure fairness:

## Algorithm 1 EWC-TTA Test-Time Adaptation

---

```

1: Input: Base encoder  $f_{\theta_{\text{pre}}}$ , task vectors  $\{\tau_k\}_{k=1}^K$ ,
   expert encoders  $\{f_{\theta_k}\}_{k=1}^K$ , expert heads  $\{h_{\phi_k}\}_{k=1}^K$ ,
   diagonal Fisher matrices  $\{\mathbf{F}_k\}_{k=1}^K$ , regularization
   coefficient  $\gamma$ , head learning rate  $\eta_{\text{head}}$ , coefficient
   learning rate  $\eta_{\lambda}$ .
2: Initialize: Merging coefficients  $\lambda \leftarrow [1/K, \dots, 1/K]^T$ , active classification heads
    $h_{\phi_k} \leftarrow \text{copy}(h_{\phi_k})$  for all  $k$ .
3: for each incoming test batch  $\mathcal{B}_t = \{x_i\}$  belonging
   to task  $k$  do
4:   // 1. Expert Guidance Generation
5:   Compute soft predictions:  $P_{\text{expert}} \leftarrow \text{Softmax}(h_{\phi_k}(f_{\theta_k}(x_i)))$ .
6:   // 2. Differentiable Virtual Merging
7:   Formulate virtual merged parameters:
8:    $\Theta(\lambda) \leftarrow \theta_{\text{pre}} + \sum_{k=1}^K \lambda_k \tau_k$ .
9:   Execute stateless forward pass:
10:   $z_i \leftarrow \text{functional\_call}(f_{\theta_{\text{pre}}}, \Theta(\lambda), x_i)$ .
11:  Compute active merged predictions:
12:   $P_{\text{merged}} \leftarrow \text{Softmax}(\tilde{h}_{\phi_k}(z_i))$ .
13:  // 3. Loss Calculation
14:   $\mathcal{L}_{\text{KL}} \leftarrow \text{DKL}(P_{\text{expert}} || P_{\text{merged}})$ .
15:   $\mathcal{L}_{\text{EWC}} \leftarrow \frac{1}{2} \sum_p F_{k,p} (\phi_{k,p} - \phi_{k,p}^{\text{init}})^2$ .
16:   $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{KL}} + \gamma \mathcal{L}_{\text{EWC}}$ .
17:  // 4. Optimization Step
18:  Compute gradients:  $\nabla_{\phi_k} \mathcal{L}_{\text{total}}, \nabla_{\lambda} \mathcal{L}_{\text{total}}$ .
19:  Update active head:  $\phi_k \leftarrow \phi_k - \eta_{\text{head}} \nabla_{\phi_k} \mathcal{L}_{\text{total}}$ .
20:  Update coefficients:  $\lambda \leftarrow \lambda - \eta_{\lambda} \nabla_{\lambda} \mathcal{L}_{\text{total}}$ .
21:  Project to simplex:  $\lambda \leftarrow \text{ProjectToSimplex}(\lambda)$ .
22:  // 5. Inference
23:  Predict labels for  $\mathcal{B}_t$  using the updated parameters.
24: end for

```

---

- Optimizer: Adam with a learning rate of  $10^{-4}$  and default momentum parameters ( $\beta_1 = 0.9, \beta_2 = 0.999$ ).
- Batch Size: 128 images per batch.
- Epochs: 3 epochs, which was found to be sufficient to achieve near-optimal convergence on all three classification benchmarks.
- Loss Function: Standard cross-entropy loss.

This pre-training regimen yielded high standalone experts, ensuring that the model merging start points are strong.

## References

- Ainsworth, S. et al. Git re-basin: Merging models modulo permutation symmetries. ICML, 2022.
- Anonymous, A. Sharpness-aware test-time adaptation for low-rank model merging. Conference Submission, 4, 2026a.
- Anonymous, A. Surrogate procrustes orthogonality regularization for flatness-orthogonality alignment. Conference Submission, 5, 2026b.
- Anonymous, A. Soft-bounded fisher-guided tta for per-tensor model merging. Conference Submission, 8, 2026c.
- Bengio, Y. et al. Representation learning: A review and new perspectives. IEEE TPAMI, 2012.
- Choshen, L. et al. Fusing fine-tuned models for better generalisation. arXiv preprint, 2022.
- Clanuwat, T., Bober-Irizar, M., Kitamoto, A., Lamb, A., Yamamoto, K., and Ha, D. Deep learning for classical japanese literature with kmnist dataset. arXiv preprint arXiv:1812.01718, 2018.
- Ilharco, G., Ribeiro, M. T., Wortsman, M., Gururangan, S., Shwartz, V., Hajishirzi, H., and Farhadi, A. Editing models with task arithmetic. In International Conference on Learning Representations, 2022.
- Jordan, M. et al. Repair: Rescaling parameters to avoid loss in merging. arXiv preprint, 2024.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences, 114(13):3521–3526, 2017.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11):2278–2324, 1998.
- Li, Y. et al. Collaborative test-time adaptation with parameter-efficient model merging. ECCV, 2024.
- Matena, M. and Raffel, C. Merging models with fisher weighted averaging. NeurIPS, 2021.
- Pan, S. J. and Yang, Q. A survey on transfer learning. IEEE Transactions on Knowledge and Data Engineering, 2009.

- Shao, J. et al. Test-time adaptation for deep learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- Singh, S. P. and Jaggi, M. Model fusion via optimal transport. *NeurIPS*, 2020.
- Stoica, G. et al. Zipit! merging models with disparate structures. *ICLR*, 2023.
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.
- Wang, Q., Kudva, O., Choi, M., Georgoulis, S., Ju, D., Kamal, M., and Van Gool, L. Continual test-time domain adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7201–7211, 2022.
- Wang, Y. et al. Sharpness-aware optimization for robust test-time adaptation. *ICML*, 2024.
- Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A. S., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S., et al. Model soups: averaging weights of multiple fine-tuned models improves out-of-distribution performance. In *International Conference on Machine Learning*, pp. 23965–23998. PMLR, 2022.
- Xiao, H., Rasul, K., and Vollgraf, R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.
- Yadav, P. et al. Resolving interference in single-stage multi-task learning. *NeurIPS*, 2023.
- Yosinski, J. et al. How transferable are features in deep neural networks? *NeurIPS*, 2014.
- Zhang, J. et al. A survey on model merging in deep learning. *arXiv preprint*, 2022.
- Zhao, T. et al. Test-time model merging for diverse domain generalization. *CVPR*, 2024.